

B1

B2

M3

R4

B5

M6

TECHNICAL DESIGN DOCUMENT

Cloning Glofox's Room Creation Feature

Rooms, layouts & spot booking — a full-stack replication of
ABC Glofox's facility management and spot-booking system.

Django 5.x · Django REST Framework · Celery · PostgreSQL 15+ · Redis · React
Native

DOCUMENT

Engineering Design Spec

STACK

DRF Backend + RN Frontend

STATUS

v1.0 — For Build

Table of Contents

01	What We're Cloning	3
02	High-Level Architecture	4
03	Data Model (ERD)	5
04	API Specification (DRF)	8
05	Backend Implementation (Django)	12
06	Frontend Implementation (React Native)	18
07	Business Rules Checklist	22
08	End-to-End Flows	23
09	Testing Plan	25
10	Build Order (Suggested Sprints)	26
11	Gotchas & Decisions to Make Early	27

1 What We're Cloning

Glofox (ABC Glofox) lets studio owners create **Rooms / Facilities** and **Room Layouts** that power its scheduling and spot-booking system. The feature set we're replicating:

Capability	Description
Room CRUD	Create, edit, delete, and restore rooms/facilities per location
Room Layout Builder	A grid editor where operators drag/drop "spots" (bikes, mats, reformers) to design the physical room
Spot Types	Custom spot categories (e.g., Bike, Mat, Reformer) with prefix, numbering, and bookable/non-bookable behavior (max ~20 per location)
Capacity Validation	Layout capacity auto-computed from spots; validated against class size
Layout → Class Assignment	Assign a room/layout to scheduled classes; changes propagate to future occurrences
Spot Booking	Members pick an exact spot from an interactive map during booking; auto-assignment for guests/web flows
Spot Blocking	Block/unblock individual spots per class occurrence
Soft-delete & Restore	Deleted layouts can be restored; existing bookings are preserved on layout edits

2 High-Level Architecture

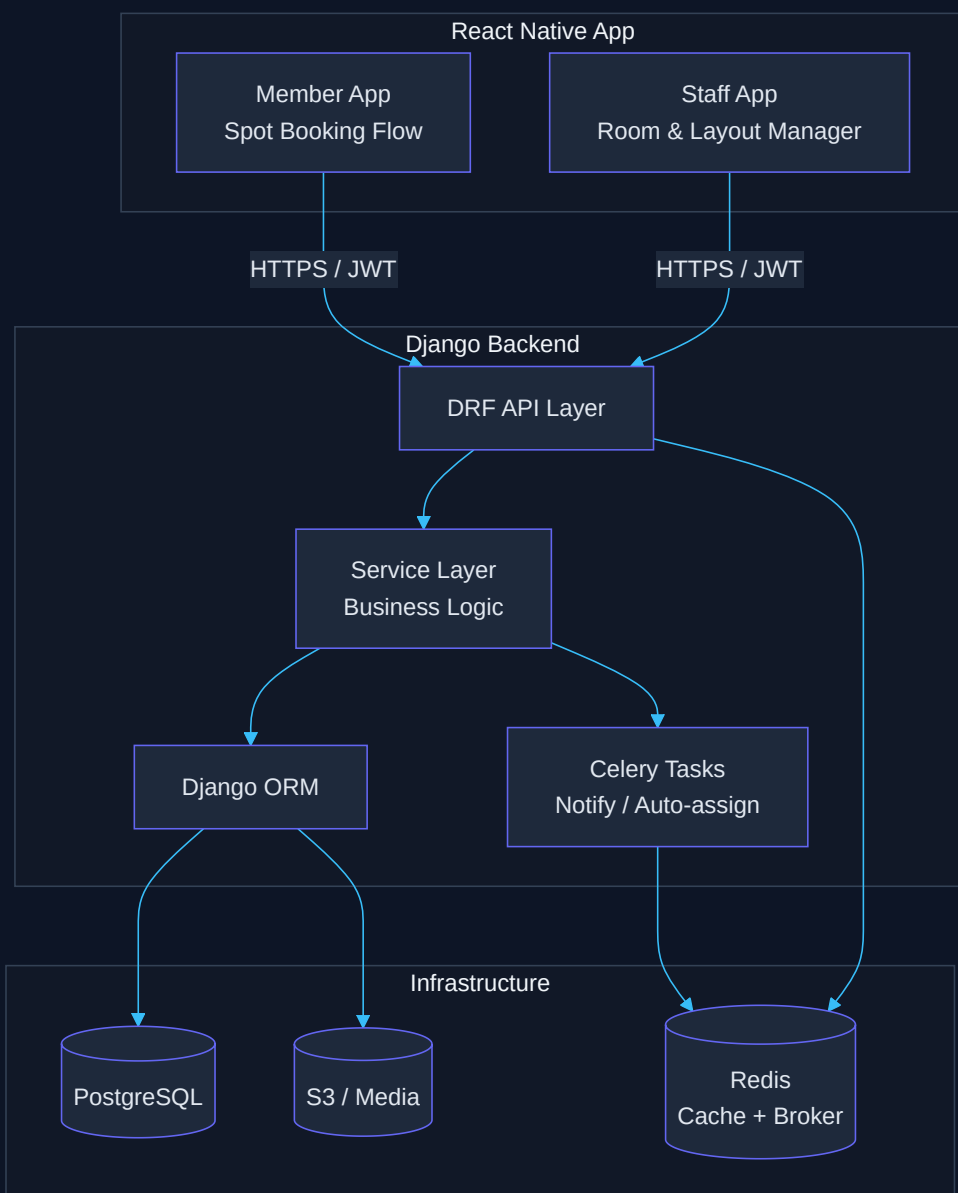


FIGURE 1 High-level architecture — React Native clients, Django backend, infrastructure

Key decisions

- **PostgreSQL** — relational integrity for spots/bookings + **JSONB** for flexible layout grid data.
- **DRF + SimpleJWT** — token auth for the React Native app.
- **Celery** — async spot auto-assignment, notification fan-out when layouts change.
- **Soft deletes everywhere** — Glofox lets operators restore deleted layouts; hard deletes would destroy booking history.

3 Data Model (ERD)

3.1 Facilities domain

Locations own rooms and spot types; rooms contain versioned layouts; layouts contain individually-addressable spots.

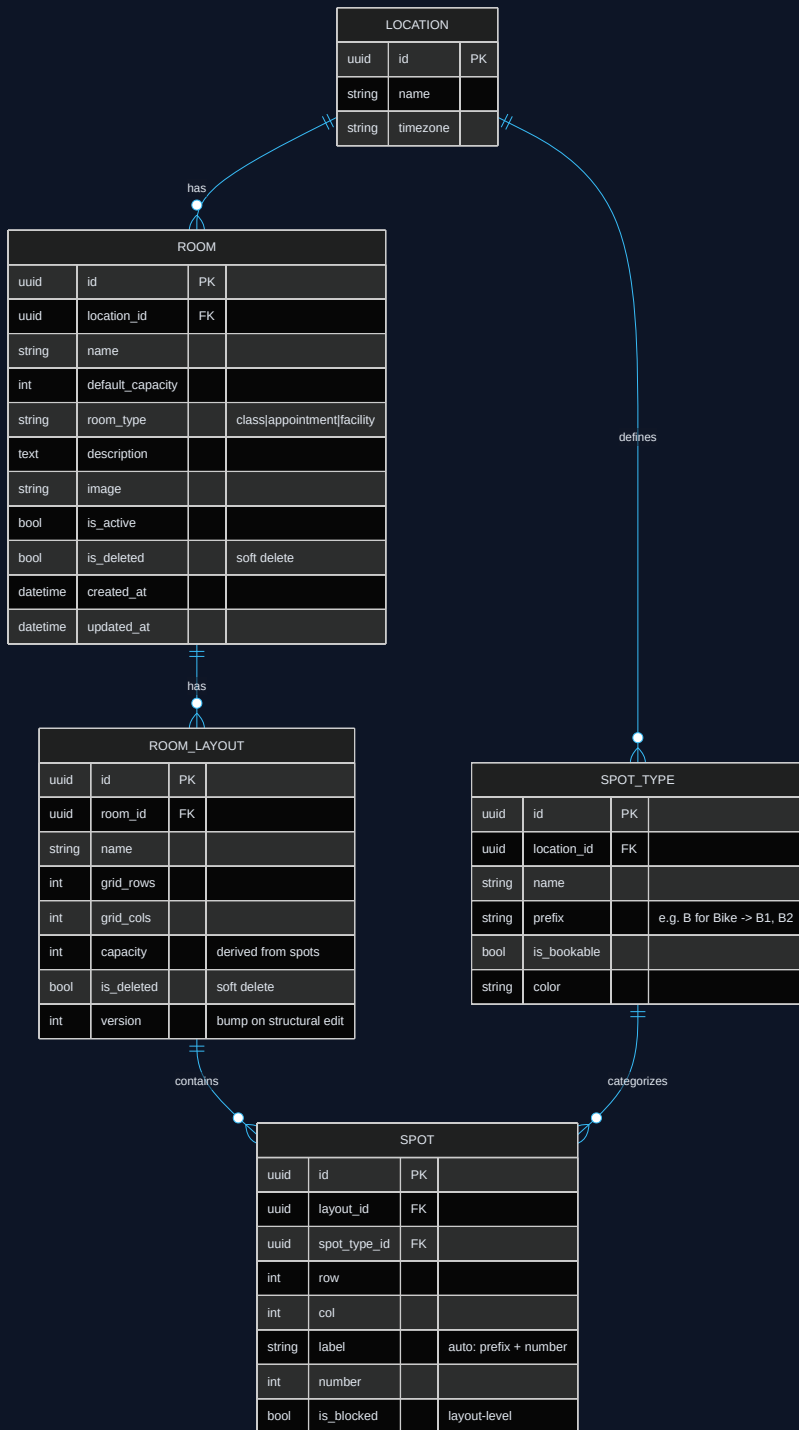


FIGURE 2 Facilities domain — locations, rooms, layouts, spot types, spots

3.2 Scheduling & booking domain

Class events pin a layout version, bookings attach members to spots. (Room, RoomLayout and Spot attributes are shown in Figure 2.)

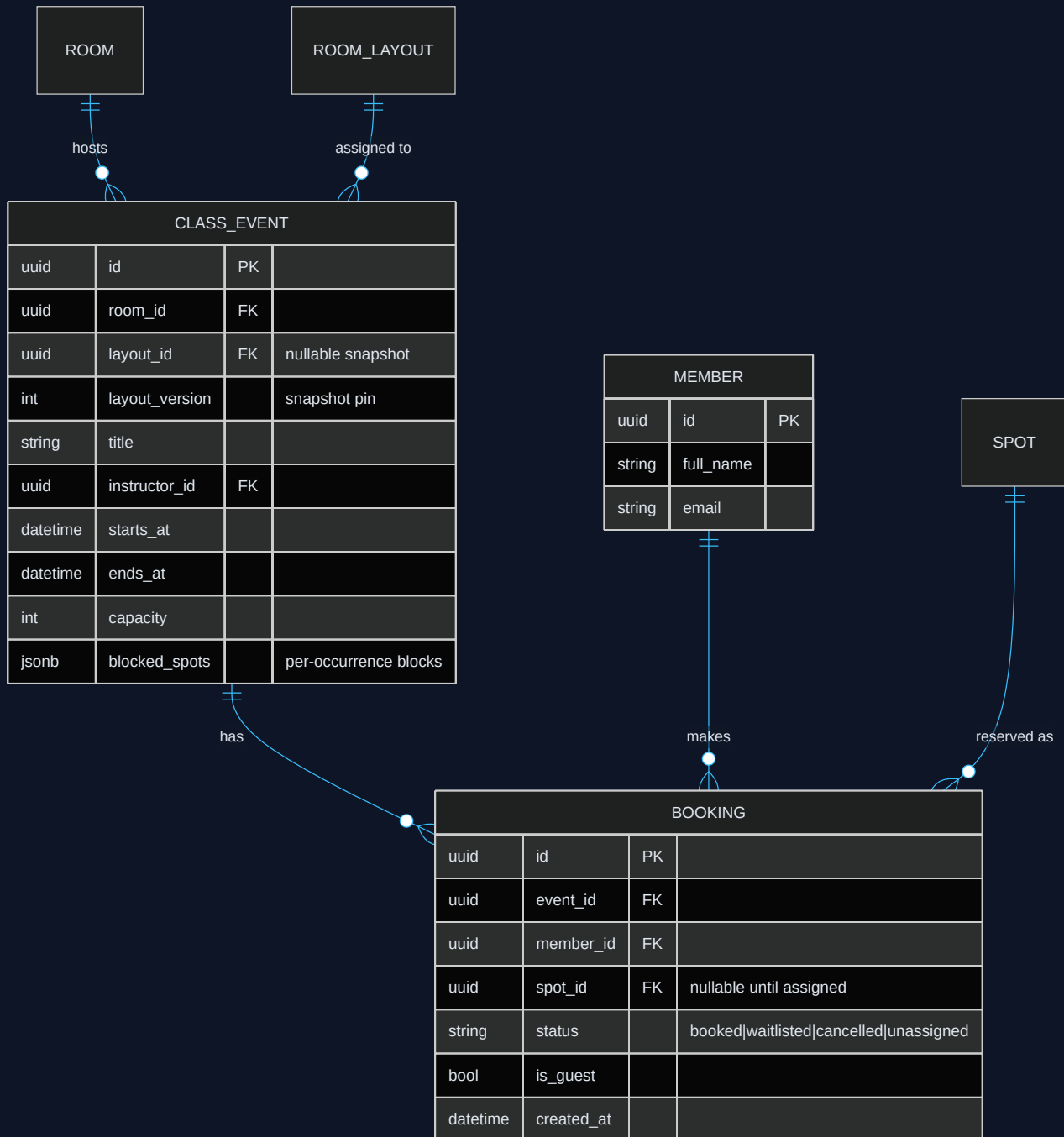


FIGURE 3 Scheduling & booking domain — class events, bookings, members

3.3 Design notes

1. **Layout versioning:** when an operator edits a layout, `version` increments. Existing `CLASS_EVENT` s pin the version they were scheduled with, so past bookings are never corrupted (mirrors Glofox's "existing bookings are preserved" behavior).
2. `blocked_spots` as JSONB on the event: per-occurrence spot blocks (e.g., a broken bike tonight only) shouldn't mutate the shared layout.
3. **Capacity is derived:** `ROOM_LAYOUT.capacity = COUNT(spots WHERE is_bookable)` . Validated server-side against `CLASS_EVENT.capacity` .
4. **Spot numbering is never renumbered** after removal (Glofox behavior) — gaps are fine and keep labels stable.

4 API Specification (DRF)

All endpoints live under `/api/v1/`. Room and layout management is staff-only; spot maps are readable by authenticated members.

Room Endpoints

POST `/locations/{id}/rooms/`
— create room

GET `/locations/{id}/rooms/` —
list rooms

GET `/rooms/{id}/` — room
detail

PATCH `/rooms/{id}/` —
update room

DELETE `/rooms/{id}/` — soft
delete

POST `/rooms/{id}/restore/` —
restore

FIGURE 4 Room endpoints

Layout Endpoints

POST `/rooms/{id}/layouts/` —
create layout + spots

GET `/layouts/{id}/` — layout +
spot grid

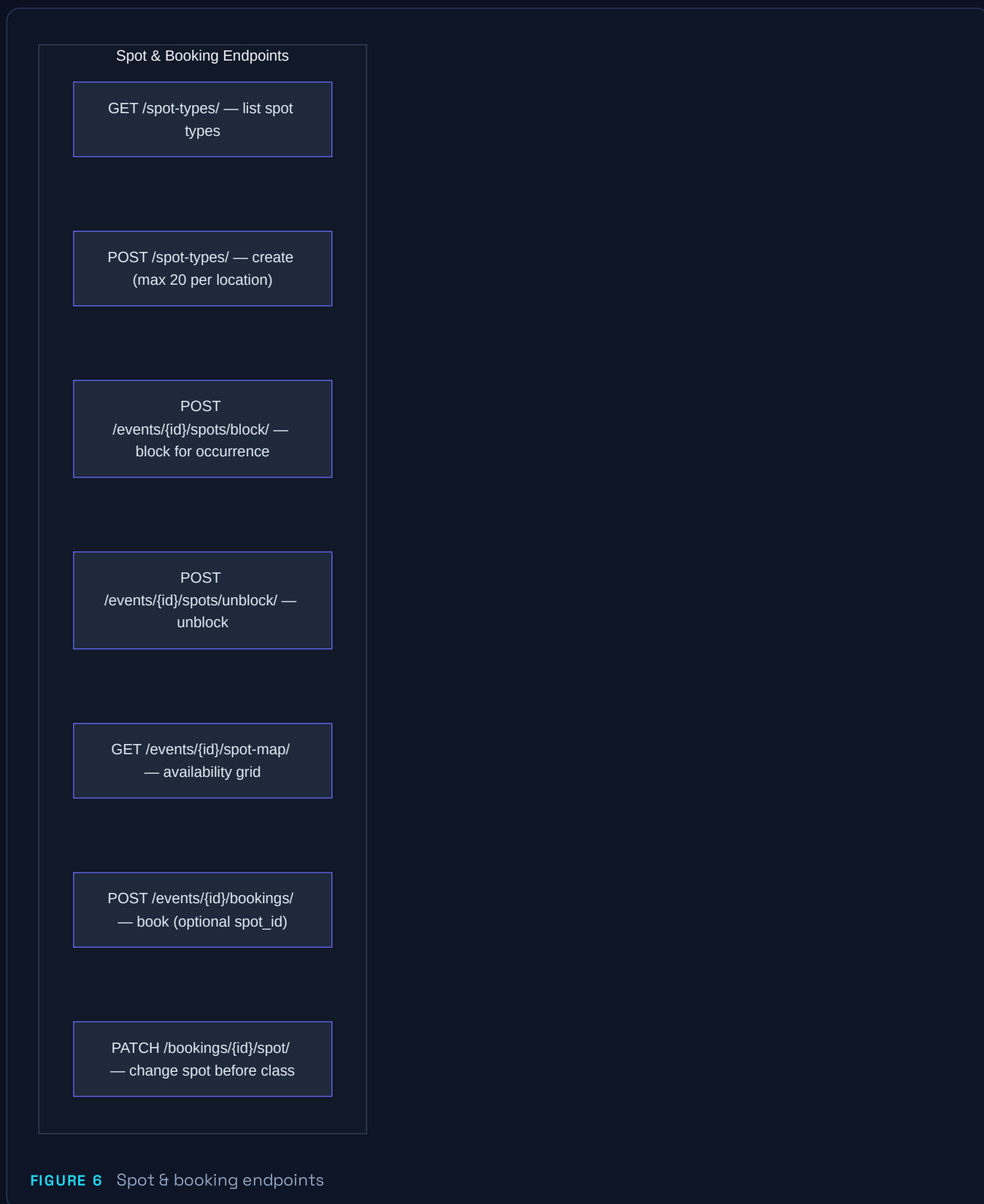
PATCH `/layouts/{id}/` — edit
grid (bumps version)

DELETE `/layouts/{id}/` — soft
delete

POST `/layouts/{id}/restore/`
— restore

GET `/layouts/{id}/classes/` —
classes using layout

FIGURE 5 Layout endpoints



4.1 Request / response examples

Create a room

```
POST /api/v1/locations/9f2c.../rooms/
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "name": "Cycle Studio A",
  "room_type": "class",
  "default_capacity": 30,
  "description": "Basement cycling studio",
  "image": "<multipart upload or S3 key>"
}
```

Create a layout with spots (grid editor payload)

```
POST /api/v1/rooms/{room_id}/layouts/
{
  "name": "Standard 30-bike layout",
  "grid_rows": 6,
  "grid_cols": 6,
  "spots": [
    { "row": 0, "col": 2, "spot_type": "bike" },
    { "row": 0, "col": 3, "spot_type": "bike" },
    { "row": 5, "col": 2, "spot_type": "instructor", "bookable_override": false }
  ]
}
```

Response

```
{
  "id": "7a1e...",
  "name": "Standard 30-bike layout",
  "grid_rows": 6,
  "grid_cols": 6,
  "capacity": 35,
  "version": 1,
  "spots": [
    { "id": "s-01", "row": 0, "col": 2, "label": "B1", "spot_type": "bike", "is_bookable": true }
  ]
}
```

Spot map for booking (member app)

```
GET /api/v1/events/{event_id}/spot-map/

{
  "layout": { "grid_rows": 6, "grid_cols": 6, "name": "Standard 30-bike layout" },
  "spots": [
    { "id": "s-01", "row": 0, "col": 2, "label": "B1", "state": "available" },
    { "id": "s-02", "row": 0, "col": 3, "label": "B2", "state": "booked" },
  ]
}
```

```
], "my_spot": "s-09"}
```

SERVER-SIDE STATE

`state` is computed server-side: `available` | `booked` | `blocked` | `non_bookable` | `mine`. The client never derives availability itself.

5 Backend Implementation (Django)

5.1 Project layout

```

backend/
├── config/
│   ├── settings.py
│   └── urls.py
├── facilities/                # the room feature app
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── services.py           # business logic (keep views thin)
│   ├── permissions.py
│   ├── tasks.py              # celery
│   └── tests/
├── scheduling/               # classes & bookings (existing or stubbed)
└── accounts/

```

5.2 Models (facilities/models.py)

```

import uuid
from django.db import models
from django.core.exceptions import ValidationError

class SoftDeleteQuerySet(models.QuerySet):
    def alive(self):
        return self.filter(is_deleted=False)

class SoftDeleteModel(models.Model):
    is_deleted = models.BooleanField(default=False)
    deleted_at = models.DateTimeField(null=True, blank=True)

    objects = SoftDeleteQuerySet.as_manager()

    class Meta:
        abstract = True

    def delete(self, *args, **kwargs):
        from django.utils import timezone
        self.is_deleted = True
        self.deleted_at = timezone.now()
        self.save(update_fields=["is_deleted", "deleted_at"])

    def restore(self):
        self.is_deleted = False
        self.deleted_at = None
        self.save(update_fields=["is_deleted", "deleted_at"])

class Room(SoftDeleteModel):
    ROOM_TYPES = [("class", "Class"), ("appointment", "Appointment"), ("facility", "Facility")]

```

```

id= models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    location = models.ForeignKey("accounts.Location", on_delete=models.CASCADE,
related_name="rooms")
    name = models.CharField(max_length=120)
    room_type = models.CharField(max_length=20, choices=ROOM_TYPES, default="class")
    default_capacity = models.PositiveIntegerField(default=0)
    description = models.TextField(blank=True)
    image = models.ImageField(upload_to="rooms/", blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
class Meta:
    unique_together = [("location", "name")]# optional: Glofox doesn't enforce it
    indexes = [models.Index(fields=["location", "is_deleted"])]
class SpotType(models.Model):
id= models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    location = models.ForeignKey("accounts.Location", on_delete=models.CASCADE,
related_name="spot_types")
    name = models.CharField(max_length=60)
    prefix = models.CharField(max_length=6)
    is_bookable = models.BooleanField(default=True)
    color = models.CharField(max_length=7, default="#6366F1")

MAX_PER_LOCATION = 20
def clean(self):
    qs = SpotType.objects.filter(location=self.location)
    if not self.pk and qs.count() >= self.MAX_PER_LOCATION:
        raise ValidationError("Maximum of 20 spot types per location.")
class RoomLayout(SoftDeleteModel):
id= models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    room = models.ForeignKey(Room, on_delete=models.CASCADE, related_name="layouts")
    name = models.CharField(max_length=120)
    grid_rows = models.PositiveSmallIntegerField()
    grid_cols = models.PositiveSmallIntegerField()
    version = models.PositiveIntegerField(default=1)
@property
def capacity(self):
    return self.spots.filter(spot_type__is_bookable=True).count()
class Spot(models.Model):
id= models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    layout = models.ForeignKey(RoomLayout, on_delete=models.CASCADE, related_name="spots")
    spot_type = models.ForeignKey(SpotType, on_delete=models.PROTECT)
    row = models.PositiveSmallIntegerField()
    col = models.PositiveSmallIntegerField()
    number = models.PositiveIntegerField()# stable, never renumbered
    label = models.CharField(max_length=12, editable=False)
    is_blocked = models.BooleanField(default=False)
class Meta:
    unique_together = [("layout", "row", "col"), ("layout", "number")]
def save(self, *args, **kwargs):
    self.label = f"{self.spot_type.prefix}{self.number}"
    super().save(*args, **kwargs)

```

5.3 Serializers (facilities/serializers.py)

```

from rest_framework import serializers
from .models import Room, RoomLayout, Spot, SpotType

class RoomSerializer(serializers.ModelSerializer):
    class Meta:
        model = Room
        fields = ["id", "name", "room_type", "default_capacity",
            "description", "image", "is_active", "created_at"]
        read_only_fields = ["id", "created_at"]

class SpotInputSerializer(serializers.Serializer):

```

```

row = serializers.IntegerField(min_value=0)
col = serializers.IntegerField(min_value=0)
spot_type = serializers.SlugRelatedField(slug_field="name",

queryset=SpotType.objects.all())classSpotSerializer(serializers.ModelSerializer):
    spot_type = serializers.SlugRelatedField(slug_field="name", read_only=True)classMeta:
        model = Spot

        fields =

["id", "row", "col", "label", "number", "spot_type", "is_blocked"]classRoomLayoutSerializer(serializers.M
odelSerializer):
    spots = SpotSerializer(many=True, read_only=True)
    capacity = serializers.IntegerField(read_only=True)classMeta:
        model = RoomLayout

        fields =

["id", "room", "name", "grid_rows", "grid_cols", "capacity", "version", "spots"]classRoomLayoutCreateSeria
lizer(serializers.ModelSerializer):
    spots = SpotInputSerializer(many=True, write_only=True)classMeta:
        model = RoomLayout
        fields = ["id", "room", "name", "grid_rows", "grid_cols", "spots"]defvalidate_spots(self, spots):
    seen =set()for s in spots:
        key =(s["row"], s["col"])if key in seen:raise serializers.ValidationError("Duplicate
spot position.")
        seen.add(key)return spots

```

5.4 Service layer (facilities/services.py)

Keep transactions and the tricky rules here, not in views.

```

from django.db import transaction
from .models import RoomLayout, Spot

@transaction.atomic
def create_layout(*, room, name, grid_rows, grid_cols, spots_data) -> RoomLayout:
    layout = RoomLayout.objects.create(
        room=room, name=name, grid_rows=grid_rows, grid_cols=grid_cols
    )
    counters = {} # per spot-type numbering, never reused within layout
    for s in spots_data:
        st = s["spot_type"]
        counters[st.id] = counters.get(st.id, 0) + 1
        Spot.objects.create(layout=layout, spot_type=st,
                            row=s["row"], col=s["col"], number=counters[st.id])

    return layout

@transaction.atomic
def update_layout(*, layout, spots_data) -> RoomLayout:
    """
    Structural edit:
    1. bump layout.version
    2. future ClassEvents pick up the new version
    3. bookings on old spots that no longer exist -> status='unassigned'
    4. celery task auto-reassigns unassigned bookings
    """

    from scheduling.models import Booking
    from facilities.tasks import auto_assign_unassigned

```

```

layout.version +=1
layout.save(update_fields=["version"])

old_spot_ids =set(layout.spots.values_list("id", flat=True))
layout.spots.all().delete()
new_spot_ids =set()
counters ={}for s in spots_data:
    st = s["spot_type"]
    counters[st.id]= counters.get(st.id,0)+1
    spot = Spot.objects.create(layout=layout, spot_type=st,
                               row=s["row"], col=s["col"], number=counters[st.id])
    new_spot_ids.add(spot.id)

orphaned = Booking.objects.filter(
    spot_id__in=old_spot_ids - new_spot_ids, status="booked")
orphaned.update(status="unassigned", spot=None)
auto_assign_unassigned.delay(layout_id=str(layout.id))return layout

defvalidate_event_capacity(*, event, layout):"""Glofox rule: validate Room Layout capacity against
class size."""if event.capacity > layout.capacity:raise ValueError(f"Class capacity
({event.capacity}) exceeds layout capacity ({layout.capacity}).")

```

5.5 Views & URLs (facilities/views.py)

```

from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import response
from .models import Room, RoomLayout
from .serializers import RoomSerializer, RoomLayoutSerializer, RoomLayoutCreateSerializer
from .services import create_layout, update_layout
from .permissions import IsStaffForWrite

class RoomViewSet(viewsets.ModelViewSet):
    serializer_class = RoomSerializer
    permission_classes = [IsStaffForWrite]

    def get_queryset(self):
        qs = Room.objects.filter(location_id=self.kwargs["location_pk"])
        if self.request.query_params.get("include_deleted") == "1":
            return qs
        return qs.alive()

    @action(detail=True, methods=["post"])
    def restore(self, request, **kwargs):
        room = self.get_object()
        room.restore()
        return response(RoomSerializer(room).data)

class RoomLayoutViewSet(viewsets.ModelViewSet):
    permission_classes = [IsStaffForWrite]

    def get_queryset(self):
        return RoomLayout.objects.alive().prefetch_related("spots__spot_type")

```

```

if self.action == "create": return RoomLayoutCreateSerializer
    return RoomLayoutSerializer

    def perform_create(self, serializer):
        data = serializer.validated_data
        layout = create_layout(
            room=data["room"], name=data["name"],
            grid_rows=data["grid_rows"], grid_cols=data["grid_cols"],
            spots_data=data["spots"],)
        serializer.instance = layout

    @action(detail=True, methods=["get"]) def classes(self, request, pk=None):
        layout = self.get_object()
        events = layout.classevent_set.filter(starts_at__gte=timezone.now())
        return response(ClassEventSerializer(events, many=True).data)

```

```

# config/urls.py
router.register(r"rooms", RoomViewSet, basename="room")
router.register(r"layouts", RoomLayoutViewSet, basename="layout")
router.register(r"spot-types", SpotTypeViewSet, basename="spottype")

```

5.6 Permissions (facilities/permissions.py)

```

from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsStaffForWrite(BasePermission):
    """Members can read spot maps; only staff/owners create rooms & layouts."""
    def has_permission(self, request, view):
        if request.method in SAFE_METHODS:
            return request.user.is_authenticated
        return request.user.is_authenticated and request.user.role in ("staff", "owner")

```

5.7 Booking-time spot logic (sequence)

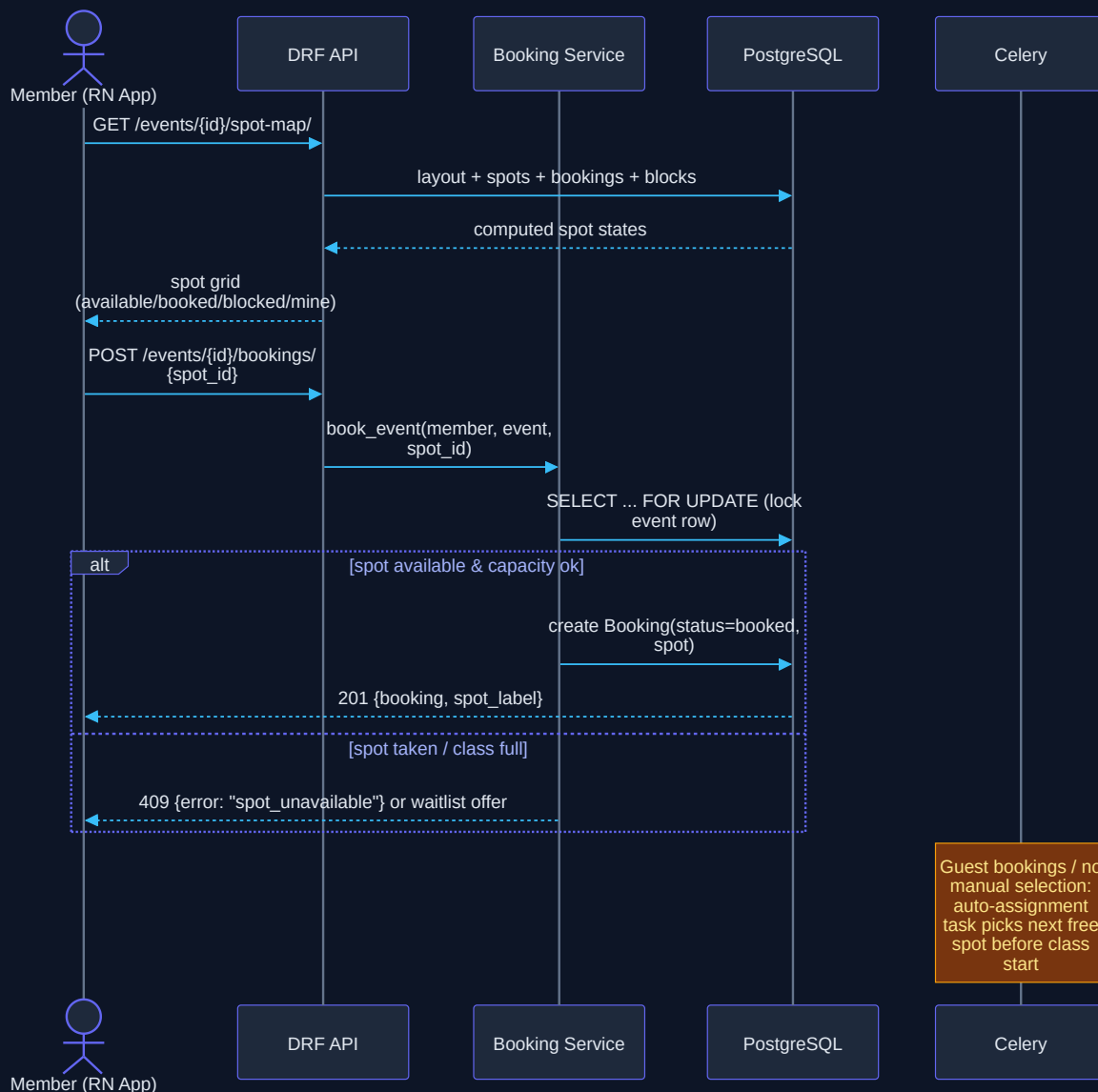


FIGURE 7 Booking-time spot logic — member spot selection with row locking

CONCURRENCY RULE — IMPORTANT

Two members tapping the same spot simultaneously must not double-book. Wrap booking in `transaction.atomic()` + `select_for_update()` on the event row, and enforce a **partial unique constraint**:

```

class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["event", "spot"],
            condition=models.Q(status="booked"),
            name="unique_active_spot_booking",
        )
    ]
  
```

6 Frontend Implementation (React Native)

6.1 Screen map

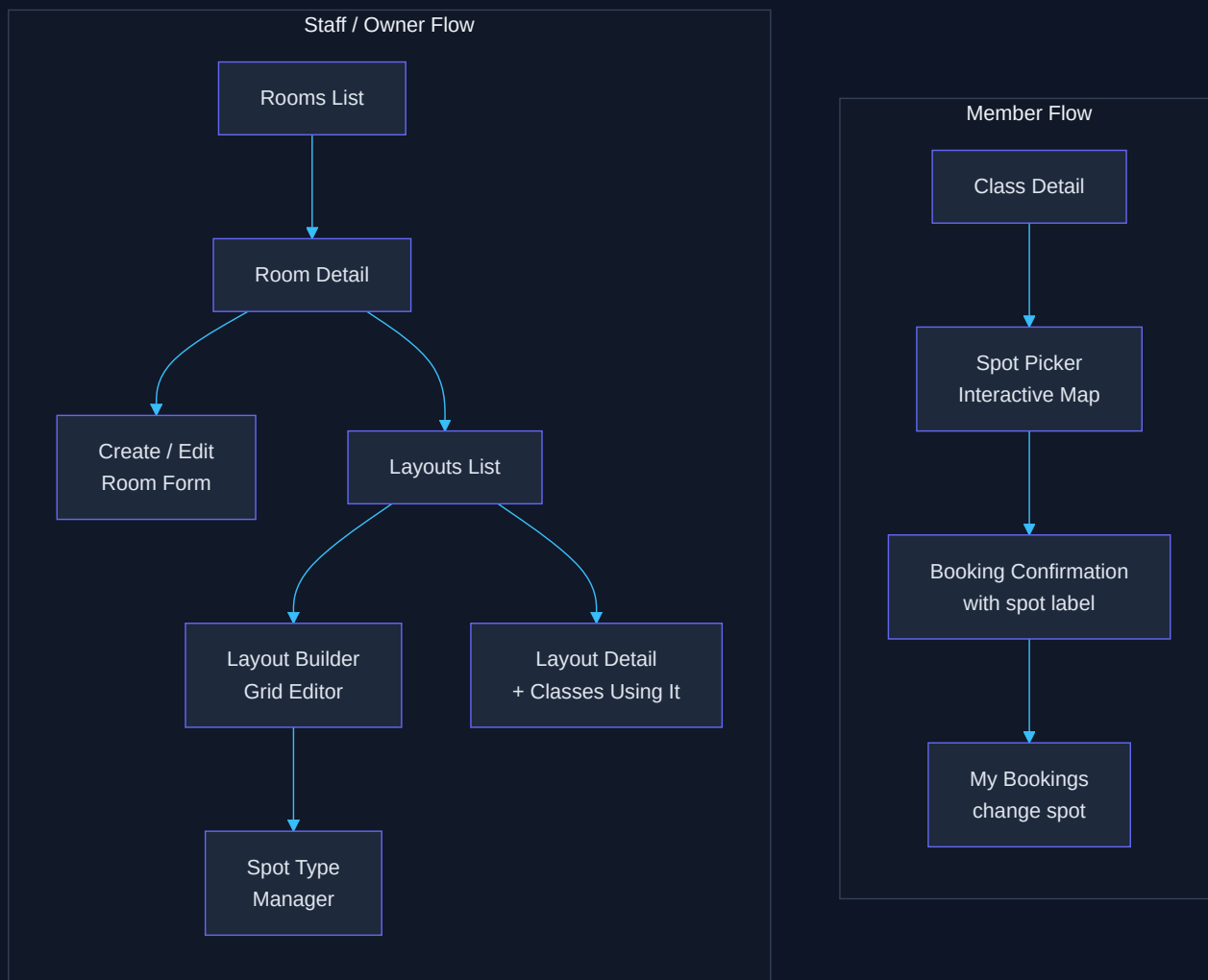


FIGURE 8 Screen map — staff/owner flow and member flow

6.2 Component tree

```

src/
├── features/
│   └── rooms/
│       ├── api/
│       │   ├── roomsApi.ts           # RTK Query or react-query hooks
│       │   └── types.ts
│       └── screens/
│           ├── RoomsListScreen.tsx
│           ├── RoomFormScreen.tsx
│           ├── LayoutBuilderScreen.tsx # the grid editor
│           ├── SpotTypeManagerScreen.tsx
│           └── SpotPickerScreen.tsx    # member-facing map
  
```

```

├── components/
│   ├── RoomCard.tsx
│   ├── LayoutGrid.tsx           # reusable grid renderer
│   ├── DraggableSpot.tsx       # builder mode
│   ├── SpotCell.tsx            # picker mode
│   └── SpotTypeChip.tsx
├── hooks/
│   ├── useLayoutBuilder.ts      # grid state machine
│   └── useSpotMap.ts           # polling / refresh
└── store/
    └── layoutBuilderSlice.ts

```

6.3 The Layout Builder (grid editor) — core logic

This is the hardest UI piece. State machine:

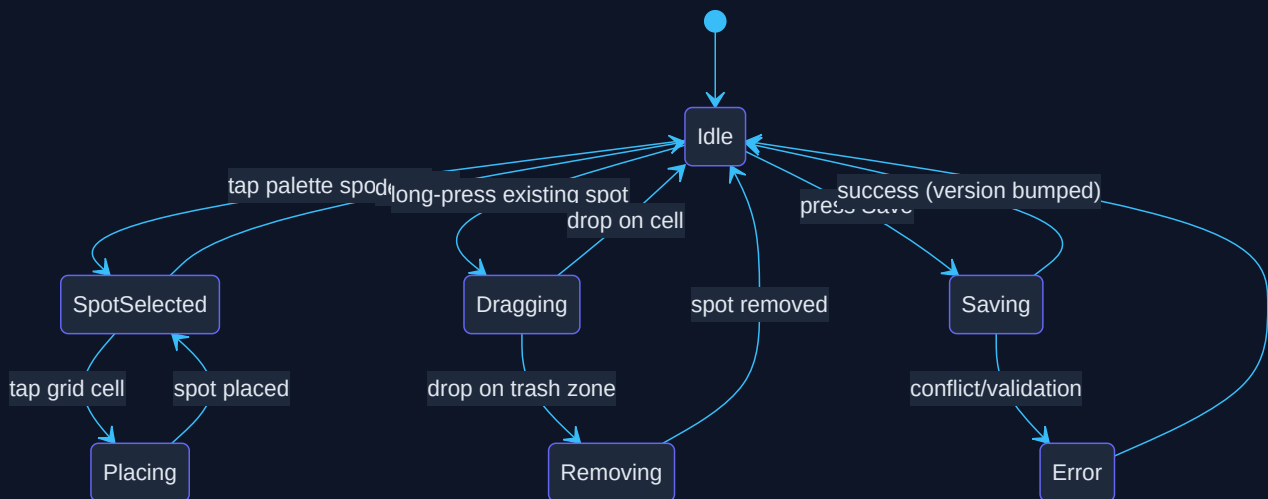


FIGURE 9 Layout builder state machine

```

// useLayoutBuilder.ts - simplified
interface BuilderSpot { row: number; col: number; typeId: string; tempId: string }

export function useLayoutBuilder(layoutId?: string) {
  const [rows, setRows] = useState(6);
  const [cols, setCols] = useState(6);
  const [spots, setSpots] = useState<BuilderSpot[]>([]);
  const [activeType, setActiveType] = useState<string | null>(null);

  const placeSpot = (row: number, col: number) => {
    if (!activeType) return;
    if (spots.some(s => s.row === row && s.col === col)) return; // occupied
    setSpots(prev => [...prev, { row, col, typeId: activeType, tempId: nanoid() }]);
  };

  const removeSpot = (tempId: string) =>
    setSpots(prev => prev.filter(s => s.tempId !== tempId));

  const save = async () => {
    const payload = {

```

```

    name, grid_rows: rows, grid_cols: cols,
    spots: spots.map(s =>({ row: s.row, col: s.col, spot_type: s.typeId })),);return layoutId
    ? api.updateLayout(layoutId, payload)// server bumps version: api.createLayout(roomId,
    payload););return{ rows, cols, spots, activeType, setActiveType, placeSpot, removeSpot, save };}

```

```

// LayoutGrid.tsx — renders in both builder & picker modes
export function LayoutGrid({ rows, cols, renderCell }) {
  return (
    <ScrollView horizontal>
      <ScrollView>
        <View style={{ width: cols * CELL, height: rows * CELL }}>
          {Array.from({ length: rows }).map((_, r) => (
            <View key={r} style={{ flexDirection: "row" }}>
              {Array.from({ length: cols }).map((_, c) => renderCell(r, c))}
            </View>
          ))}
        </View>
      </ScrollView>
    </ScrollView>
  );
}

```

LARGE LAYOUTS

Glofox supports horizontal + vertical scrolling around large layouts — the nested `ScrollView` above covers that.

6.4 Member Spot Picker

```

// SpotPickerScreen.tsx — simplified
export default function SpotPickerScreen({ route }) {
  const { eventId } = route.params;
  const { data, refetch } = useSpotMap(eventId); // GET /events/{id}/spot-map/
  const [selected, setSelected] = useState<string | null>(null);
  const bookMutation = useBookSpot(eventId);

  const stateColor = { available: "#22C55E", booked: "#334155",
    blocked: "#F59E0B", mine: "#6366F1", non_bookable: "#94A3B8" };

  return (
    <View>
      <LayoutGrid
        rows={data.layout.grid_rows}
        cols={data.layout.grid_cols}
        renderCell={(r, c) => {
          const spot = findSpot(data.spots, r, c);
          if (!spot) return <EmptyCell key={` ${r}-${c}`} />;
          const isSelected = selected === spot.id;
          return (
            <Pressable
              key={spot.id}
              disabled={spot.state !== "available"}

```

```

style={styles.cell,
      { backgroundColor: stateColor[spot.state] },
      isSelected && styles.selected}}>
  <Text style={styles.label}>{spot.label}</Text>
</Pressable>);}}
/>
<Button
  title={selected ? `Book spot ${labelOf(selected)}`: "Select a spot"}
  disabled={!selected || bookMutation.isPending}
  onPress={()=>
    bookMutation.mutate({ spot_id: selected }, {onError:(e)=>{if(e.status ===409){// spot
sniped - refresh & notify
      Alert.alert("That spot was just taken");refetch();}},)}}
  />
</View>);}

```

6.5 API layer (react-query example)

```

// roomsApi.ts
import axios from "axios";
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";

const api = axios.create({ baseURL: "https://api.yourapp.com/api/v1" });
// attach JWT via interceptor from your auth store

export const useRooms = (locationId: string) =>
  useQuery({
    queryKey: ["rooms", locationId],
    queryFn: () => api.get(`/locations/${locationId}/rooms/`).then(r => r.data),
  });

export const useCreateRoom = (locationId: string) => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (body: RoomInput) => api.post(`/locations/${locationId}/rooms/`, body),
    onSuccess: () => qc.invalidateQueries({ queryKey: ["rooms", locationId] }),
  });
};

export const useSpotMap = (eventId: string) =>
  useQuery({
    queryKey: ["spot-map", eventId],
    queryFn: () => api.get(`/events/${eventId}/spot-map/`).then(r => r.data),
    refetchInterval: 15_000, // keep availability fresh
  });

```

7 Business Rules Checklist (parity with Glofox)

Rule	Where enforced
Max 20 custom spot types per location	<code>SpotType.clean()</code> + serializer validation
Spot labels = prefix + number (B1, M3...)	<code>Spot.save()</code>
Spot numbers never renumbered after removal	Numbering counter in service layer, gaps allowed
Layout capacity = count of bookable spots	<code>RoomLayout.capacity</code> property
Layout capacity validated against class capacity	<code>validate_event_capacity()</code> at class creation/edit
Layout edits apply to future classes only; existing bookings preserved	<code>version</code> pinning on <code>ClassEvent</code>
Members affected by structural edits → temporarily <code>unassigned</code> → auto-reassigned	<code>update_layout()</code> + <code>auto_assign_unassigned</code> Celery task
Auto-assignment for guests, web bookings, no-selection flows	Celery task before class start
Deleted rooms/layouts restorable	<code>SoftDeleteModel</code>
Per-occurrence spot blocks don't mutate the shared layout	<code>ClassEvent.blocked_spots</code> JSONB
Room Layout capacity does not auto-update class capacity	Intentional no-op (Glofox limitation we're keeping)

8 End-to-End Flows

8.1 Staff creates a room + layout

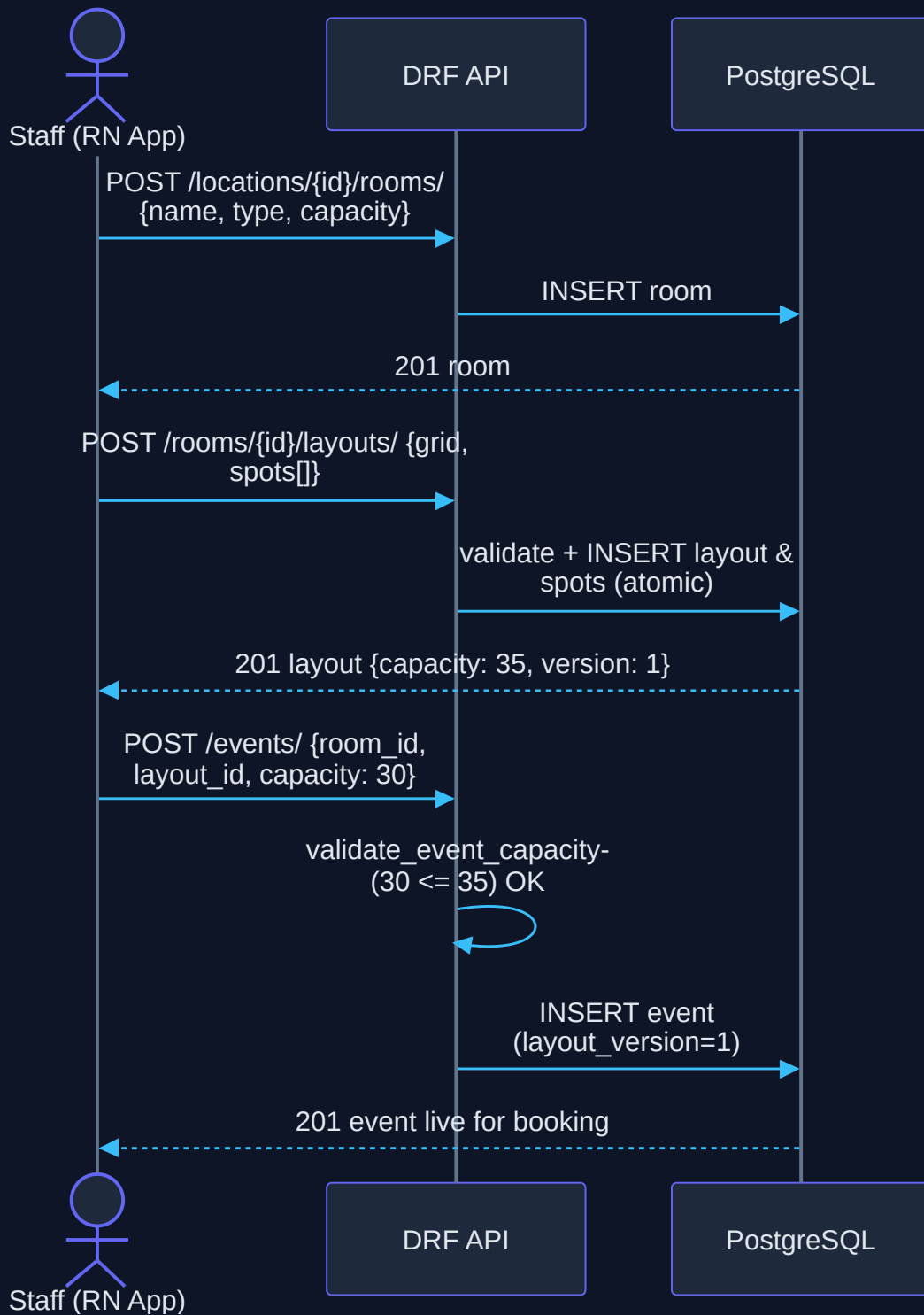


FIGURE 10 Staff creates a room, layout, and class — end to end

8.2 Operator edits layout mid-season



FIGURE 11 Mid-season layout edit — versioning, orphan handling, member notification

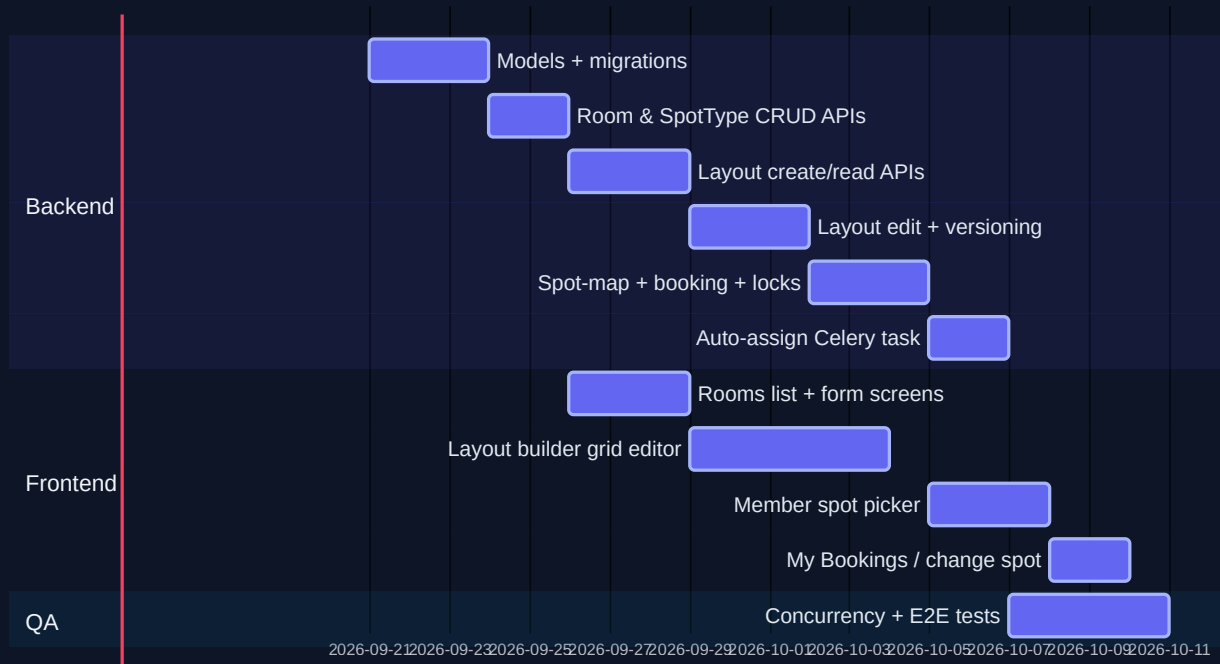
9

Testing Plan

Layer	Tests
Models	Soft delete/restore, spot label generation, 20-type cap, unique constraints
Services	Layout create/update atomicity, version bump, orphan-booking reassignment, capacity validation
API	Permission matrix (member vs staff), duplicate grid position → 400, restore endpoint, spot-map state computation
Concurrency	Two simultaneous bookings on same spot → exactly one 201, one 409
RN (Jest/RTL)	Grid renders rows×cols, occupied-cell guard, 409 → refetch + alert, builder save payload shape
E2E	Staff creates room → layout → class; member books spot; operator edits layout; member auto-reassigned

10

Build Order (Suggested Sprints)

**FIGURE 12** Suggested sprint plan — backend, frontend, QA

Gotchas & Decisions to Make Early

1. **Nested ScrollViews on Android** — set `nestedScrollEnabled` and give the grid a fixed pixel size, or gestures fight each other.
2. **Don't compute spot state on the client** — the server must be the single source of truth for `available/booked/blocked`, or race conditions leak into the UI.
3. **Grid size limits** — cap `grid_rows` / `grid_cols` (e.g., 20×20) server-side; a 100×100 grid will kill render performance and your DB row count.
4. **Version pinning vs. snapshotting** — pinning a version number is cheap; if you need historical layout *visuals*, store a JSONB snapshot of the spot grid on the event instead.
5. **Guest (+1) bookings** — primary user picks a spot; guest spot is auto-assigned by the Celery task, never manually (Glofox behavior).
6. **Restore semantics** — restoring a room should optionally restore its layouts (cascade flag); decide this before writing the endpoint.

Stack reference: Django 5.x · djangorestframework · django-celery · PostgreSQL 15+ · Redis · React Native 0.7x · @tanstack/react-query · React Navigation